

# Experiment No. 05

## Java Collection Framework: Important Methods by Interface

Date: 18/09/2026

### ■ Objective / Aim

To study and implement the **Java Collection Framework** and demonstrate the important methods of different collection interfaces such as **Collection**, **List**, **Set**, **SortedSet**, **NavigableSet**, **Queue**, **Deque**, **Map**, **SortedMap**, **NavigableMap**, **Iterator**, and **ListIterator**.

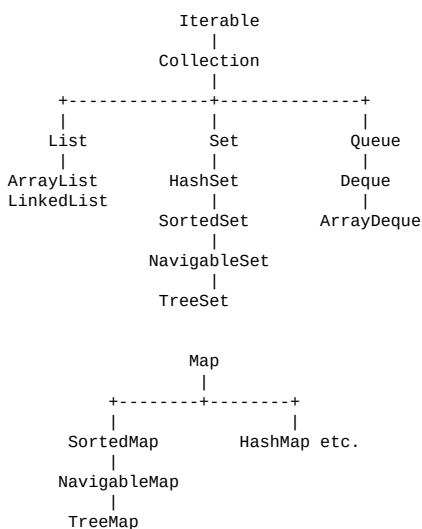
### ■■ Problem Statement

Develop a Java application to demonstrate important methods provided by the Java Collection Framework interfaces. The program should demonstrate adding, removing, searching, sizing, list access and modification, sorting, set operations, sorted and navigable operations, FIFO queue operations, double-ended queue operations, key-value mapping, sorted-map navigation, and forward/backward traversal.

### ■ Concepts Used

Java Collection Framework • Collection • List • Set • SortedSet • NavigableSet • Queue • Deque • Map • SortedMap • NavigableMap • Iterator • ListIterator • Generics • Sorting • Navigation

### ■ Collection Framework Hierarchy



Note: Map is separate from the Collection hierarchy.  
It stores data as key-value pairs.

### ■ Algorithm / Logic Description

1. Create collection objects such as ArrayList, HashSet, TreeSet, ArrayDeque, HashMap, and TreeMap.
2. Demonstrate common Collection methods such as add(), addAll(), remove(), removeAll(), contains(), containsAll(), size(), isEmpty(), clear(), and iterator().
3. Demonstrate List methods including indexed insertion, get(), set(), remove(), indexOf(), lastIndexOf(), subList(), and sort().
4. Use Set to store unique elements.
5. Use SortedSet and NavigableSet for sorted and navigation operations.
6. Use Queue methods for FIFO operations.
7. Use Deque methods to insert, remove, and access elements from both ends.
8. Use Map methods to store and retrieve key-value pairs.
9. Use SortedMap and NavigableMap to navigate sorted keys.
10. Use Iterator to traverse collection elements.
11. Use ListIterator to traverse a list in both forward and backward directions.
12. Display the results.

### ■ Important Methods by Interface

## 1. Collection Interface

add(), addAll(), remove(), removeAll(), contains(), containsAll(), size(), isEmpty(), clear(), iterator()

## 2. List Interface

*Ordered and allows duplicates.*

add(), add(index, element), get(), set(), remove(index), indexOf(), lastIndexOf(), subList(), sort()

## 3. Set Interface

*Does not guarantee insertion order and does not allow duplicate elements.*

add(), remove(), contains(), size(), isEmpty(), clear()

## 4. SortedSet Interface

Elements are maintained in sorted order. Methods: first(), last(), headSet(), tailSet(), subSet(), comparator()

## 5. NavigableSet Interface

lower(), floor(), ceiling(), higher(), pollFirst(), pollLast(), descendingSet()

## 6. Queue Interface

*FIFO - First In First Out.*

add(), offer(), remove(), poll(), element(), peek()

## 7. Deque Interface

*Double Ended Queue.*

addFirst(), addLast(), offerFirst(), offerLast(), removeFirst(), removeLast(), pollFirst(), pollLast(), peekFirst(), peekLast()

## 8. Map Interface

*Stores key-value pairs.*

put(), get(), remove(), containsKey(), containsValue(), keySet(), values(), entrySet(), size(), isEmpty(), clear()

## 9. SortedMap Interface

firstKey(), lastKey(), headMap(), tailMap(), subMap(), comparator()

## 10. NavigableMap Interface

lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry(), descendingMap()

## 11. Iterator Interface

hasNext(), next(), remove(), forEachRemaining()

## 12. ListIterator Interface

hasNext(), hasPrevious(), next(), previous(), nextIndex(), previousIndex(), add(), set(), remove()

## ■ Complete Java Source Code

```
import java.util.*;

public class CollectionFrameworkDemo {

    public static void main(String[] args) {

        // 1. COLLECTION / LIST
        List<String> list = new ArrayList<>();

        System.out.println("===== LIST / COLLECTION =====");

        list.add("Apple");
        list.add("Banana");
        list.add("Mango");
        list.add("Banana");

        System.out.println("List: " + list);
        System.out.println("Size: " + list.size());
        System.out.println("Contains Mango: " + list.contains("Mango"));

        list.addAll(Arrays.asList("Orange", "Grapes"));
        System.out.println("After addAll: " + list);

        list.remove("Banana");
        System.out.println("After remove: " + list);
        System.out.println("Is Empty: " + list.isEmpty());

        // 2. LIST METHODS
        System.out.println("\n===== LIST METHODS =====");

        list.add(1, "Pineapple");
        System.out.println("After add(index): " + list);
        System.out.println("Element at index 2: " + list.get(2));

        list.set(2, "Watermelon");
        System.out.println("After set: " + list);

        System.out.println("First index of Banana: " + list.indexOf("Banana"));
        System.out.println("Last index of Banana: " + list.lastIndexOf("Banana"));
        System.out.println("SubList: " + list.subList(1, 4));
    }
}
```

```

list.sort(String::compareTo);
System.out.println("Sorted List: " + list);

// 3. SET
System.out.println("\n===== SET =====");

Set<Integer> set = new HashSet<>();
set.add(30);
set.add(10);
set.add(20);
set.add(10);

System.out.println("Set: " + set);
System.out.println("Contains 20: " + set.contains(20));
System.out.println("Set Size: " + set.size());

set.remove(30);
System.out.println("After remove: " + set);

// 4. SORTED SET
System.out.println("\n===== SORTED SET =====");

SortedSet<Integer> sortedSet = new TreeSet<>();
sortedSet.add(50);
sortedSet.add(20);
sortedSet.add(40);
sortedSet.add(10);
sortedSet.add(30);

System.out.println("Sorted Set: " + sortedSet);
System.out.println("First: " + sortedSet.first());
System.out.println("Last: " + sortedSet.last());
System.out.println("HeadSet(30): " + sortedSet.headSet(30));
System.out.println("TailSet(30): " + sortedSet.tailSet(30));
System.out.println("SubSet(20, 50): " + sortedSet.subSet(20, 50));

// 5. NAVIGABLE SET
System.out.println("\n===== NAVIGABLE SET =====");

NavigableSet<Integer> navSet = new TreeSet<>();
navSet.add(10);
navSet.add(20);
navSet.add(30);
navSet.add(40);
navSet.add(50);

System.out.println("Navigable Set: " + navSet);
System.out.println("Lower(30): " + navSet.lower(30));
System.out.println("Floor(30): " + navSet.floor(30));
System.out.println("Ceiling(35): " + navSet.ceiling(35));
System.out.println("Higher(30): " + navSet.higher(30));
System.out.println("Descending Set: " + navSet.descendingSet());

// 6. QUEUE
System.out.println("\n===== QUEUE =====");

Queue<String> queue = new LinkedList<>();
queue.add("A");
queue.offer("B");
queue.offer("C");

System.out.println("Queue: " + queue);
System.out.println("Peek: " + queue.peek());
System.out.println("Element: " + queue.element());
System.out.println("Poll: " + queue.poll());
System.out.println("Queue after poll: " + queue);

// 7. DEQUE
System.out.println("\n===== DEQUE =====");

Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(20);
deque.addLast(30);
deque.offerFirst(10);
deque.offerLast(40);

System.out.println("Deque: " + deque);
System.out.println("First: " + deque.peekFirst());
System.out.println("Last: " + deque.peekLast());

deque.removeFirst();
deque.removeLast();

System.out.println("After removing both ends: " + deque);

// 8. MAP
System.out.println("\n===== MAP =====");

Map<Integer, String> map = new HashMap<>();
map.put(101, "Rahul");
map.put(102, "Priya");
map.put(103, "Ravi");

System.out.println("Map: " + map);
System.out.println("Value for key 101: " + map.get(101));
System.out.println("Contains key 102: " + map.containsKey(102));
System.out.println("Contains value Ravi: " + map.containsValue("Ravi"));
System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());
System.out.println("Entries: " + map.entrySet());

```

```

// 9. SORTED MAP
System.out.println("\n===== SORTED MAP =====");

SortedMap<Integer, String> sortedMap = new TreeMap<>();
sortedMap.put(103, "Ravi");
sortedMap.put(101, "Rahul");
sortedMap.put(102, "Priya");
sortedMap.put(104, "Anu");

System.out.println("Sorted Map: " + sortedMap);
System.out.println("First Key: " + sortedMap.firstKey());
System.out.println("Last Key: " + sortedMap.lastKey());
System.out.println("Head Map(103): " + sortedMap.headMap(103));
System.out.println("Tail Map(102): " + sortedMap.tailMap(102));
System.out.println("Sub Map(101,104): " + sortedMap.subMap(101, 104));

// 10. NAVIGABLE MAP
System.out.println("\n===== NAVIGABLE MAP =====");

NavigableMap<Integer, String> navMap = new TreeMap<>();
navMap.put(10, "A");
navMap.put(20, "B");
navMap.put(30, "C");
navMap.put(40, "D");
navMap.put(50, "E");

System.out.println("Navigable Map: " + navMap);
System.out.println("Lower Key(30): " + navMap.lowerKey(30));
System.out.println("Floor Key(30): " + navMap.floorKey(30));
System.out.println("Ceiling Key(35): " + navMap.ceilingKey(35));
System.out.println("Higher Key(30): " + navMap.higherKey(30));
System.out.println("First Entry: " + navMap.firstEntry());
System.out.println("Last Entry: " + navMap.lastEntry());
System.out.println("Descending Map: " + navMap.descendingMap());

// 11. ITERATOR
System.out.println("\n===== ITERATOR =====");

Iterator<String> iterator = list.iterator();

while (iterator.hasNext()) {
    System.out.println(iterator.next());
}

// 12. LIST ITERATOR
System.out.println("\n===== LIST ITERATOR =====");

List<String> names = new ArrayList<>();
names.add("Rahul");
names.add("Priya");
names.add("Ravi");

ListIterator<String> listIterator = names.listIterator();

System.out.println("Forward Traversal:");

while (listIterator.hasNext()) {
    System.out.println(listIterator.next());
}

System.out.println("Backward Traversal:");

while (listIterator.hasPrevious()) {
    System.out.println(listIterator.previous());
}
}
}

```

## ■ Compilation and Execution

**File name:** CollectionFrameworkDemo.java

```

javac CollectionFrameworkDemo.java
java CollectionFrameworkDemo

```

## ■■ Sample Output

```

===== LIST / COLLECTION =====
List: [Apple, Banana, Mango, Banana]
Size: 4
Contains Mango: true
After addAll: [Apple, Banana, Mango, Banana, Orange, Grapes]
After remove: [Apple, Mango, Banana, Orange, Grapes]
Is Empty: false

===== LIST METHODS =====
After add(index): [Apple, Pineapple, Mango, Banana, Orange, Grapes]
Element at index 2: Mango
After set: [Apple, Pineapple, Watermelon, Banana, Orange, Grapes]
First index of Banana: 3
Last index of Banana: 3
SubList: [Pineapple, Watermelon, Banana]
Sorted List: [Apple, Banana, Grapes, Orange, Pineapple, Watermelon]

===== SET =====
Set: [10, 20, 30]
Contains 20: true
Set Size: 3

```

```

After remove: [10, 20]

===== SORTED SET =====
Sorted Set: [10, 20, 30, 40, 50]
First: 10
Last: 50
HeadSet(30): [10, 20]
TailSet(30): [30, 40, 50]
SubSet(20, 50): [20, 30, 40]

===== NAVIGABLE SET =====
Navigable Set: [10, 20, 30, 40, 50]
Lower(30): 20
Floor(30): 30
Ceiling(35): 40
Higher(30): 40
Descending Set: [50, 40, 30, 20, 10]

===== QUEUE =====
Queue: [A, B, C]
Peek: A
Element: A
Poll: A
Queue after poll: [B, C]

===== DEQUE =====
Deque: [10, 20, 30, 40]
First: 10
Last: 40
After removing both ends: [20, 30]

===== MAP =====
Map: {101=Rahul, 102=Priya, 103=Ravi}
Value for key 101: Rahul
Contains key 102: true
Contains value Ravi: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Ravi]
Entries: [101=Rahul, 102=Priya, 103=Ravi]

===== SORTED MAP =====
Sorted Map: {101=Rahul, 102=Priya, 103=Ravi, 104=Anu}
First Key: 101
Last Key: 104
Head Map(103): {101=Rahul, 102=Priya}
Tail Map(102): {102=Priya, 103=Ravi, 104=Anu}
Sub Map(101,104): {101=Rahul, 102=Priya, 103=Ravi}

===== NAVIGABLE MAP =====
Navigable Map: {10=A, 20=B, 30=C, 40=D, 50=E}
Lower Key(30): 20
Floor Key(30): 30
Ceiling Key(35): 40
Higher Key(30): 40
First Entry: 10=A
Last Entry: 50=E
Descending Map: {50=E, 40=D, 30=C, 20=B, 10=A}

===== ITERATOR =====
Apple
Banana
Grapes
Orange
Pineapple
Watermelon

===== LIST ITERATOR =====
Forward Traversal:
Rahul
Priya
Ravi
Backward Traversal:
Ravi
Priya
Rahul

```

## ■ Real-Time Applications

- ArrayList → Student lists and product lists
- HashSet → Unique user IDs
- TreeSet → Sorted marks or rankings
- Queue → Printer/job scheduling
- Deque → Browser history and task processing
- HashMap → Student ID to Student Name
- TreeMap → Sorted records
- Iterator → Traversing application or database records

## ■ Result

The **Java Collection Framework** was successfully studied and implemented. The program demonstrates important methods of Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and

ListIterator interfaces and successfully performs insertion, deletion, searching, sorting, navigation, mapping, and traversal operations.

## ■ Note on Set Ordering

**Set does not guarantee insertion order and does not allow duplicate elements.** A HashSet does not guarantee order, a LinkedHashSet maintains insertion order, and a TreeSet maintains elements in sorted order.